

模板

函数模板

```
#include <iostream>
using namespace std;

// 函数模板
template <typename T>
T myMax(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    cout << myMax(3, 5) << endl;      // int 类型
    cout << myMax(2.7, 1.8) << endl;  // double 类型
    cout << myMax('a', 'z') << endl; // char 类型

    return 0;
}
```

函数模板中可以有**不止一个**类型参数

```
#include <iostream>
using namespace std;

template <typename T1, typename T2> // 类型参数表
void show(T1 a, T2 b) {
    cout << "a = " << a << ", b = " << b << endl;
}

int main() {
    show(10, 3.14);      // T1 = int, T2 = double
    show("hello", 25);   // T1 = const char*, T2 = int

    return 0;
}
```

函数模板的重载

- 函数模板可以重载，只要它们的**形参表或类型参数表**不同即可调用顺序：

1. 先找**参数完全匹配的普通函数**
2. 再找**参数完全匹配的模板函数**
3. 实参经过**自动类型转换**后能够匹配的**普通函数**
4. 报错

- 不会调用自动类型转换的模板函数

```
#include <iostream>
using namespace std;

//===== 1. 函数模板重载 =====
// 模板1: 两个参数类型相同
template <typename T>
void test(T a, T b) {
    cout << "模板1: test(T, T)" << endl;
}

// 模板2: 两个参数类型可以不同
template <typename T, typename U>
void test(T a, U b) {
    cout << "模板2: test(T, U)" << endl;
}

//===== 2. 普通函数 + 模板函数同名 =====
void func(int a, int b) {
    cout << "普通函数: func(int, int)" << endl;
}

template <typename T>
void func(T a, T b) {
    cout << "模板函数: func(T, T)" << endl;
}

//===== 3. 只有模板函数 =====
template <typename T>
void onlyTemp(T a, T b) {
    cout << "模板函数: onlyTemp(T, T)" << endl;
}

int main() {
```

```

cout << "==== 函数模板重载 =====> << endl;
test(1, 2);           // 两个都是 int
test(1, 2.5);        // int 和 double

cout << "\n==== 调用顺序演示 =====> << endl;

// 1. 先找参数完全匹配的普通函数
cout << "1. func(10, 20) -> ";
func(10, 20);

// 2. 再找参数完全匹配的模板函数
cout << "2. func('a', 'b') -> ";
func('a', 'b');

// 3. 普通函数可以靠自动类型转换匹配
// 模板 func(T, T) 这里无法把 T 同时推导成 int 和 char
cout << "3. func(10, 'a') -> ";
func(10, 'a');

// 4. 只有模板函数时, 如果需靠自动类型转换来统一 T, 则报错
// onlyTemp(10, 2.5);    // 编译错误: T 无法同时推导为 int 和 double

return 0;
}

```

类模板

```

#include <iostream>
using namespace std;

// 定义类模板
template <typename T>
class Box {
private:
    T data;

public:
    Box(T d) {
        data = d;
    }

    void show() {
        cout << "data = " << data << endl;
    }
}

```

```

    }
};

int main() {
    Box<int> b1(100);
    b1.show();

    Box<double> b2(3.14);
    b2.show();

    Box<char> b3('A');
    b3.show();

    return 0;
}

```

用类模板定义对象的写法：

类模板名<真实类型参数表> 对象名(构造函数实参表)

编译器由类模板生成类的过程叫做**类模板的实例化**，由类模板实例化得到的类，叫**模板类**

- 类模板的<类型参数表>中可以出现**非类型参数**

类模板与派生

```

#include <iostream>
using namespace std;

// T 是“类型参数”
// N 是“非类型参数”（一个整型常量）
template <typename T, int N>
class StaticArray {
private:
    // 利用传入的非类型参数 N，直接在编译期确定数组大小
    T arr[N];

public:
    // 获取数组大小
    int getSize() const {
        return N; // N 在类里面相当于一个常量 (const int)
    }

    // 设置元素
    void set(int index, T value) {

```

```

        if (index >= 0 && index < N) {
            arr[index] = value;
        } else {
            cout << "索引越界! " << endl;
        }
    }

// 获取元素
T get(int index) {
    if (index >= 0 && index < N) {
        return arr[index];
    }
    return T(); // 越界返回默认构造值
}

};

int main() {
    // 实例化时: T 被替换为 int, N 被替换为 5
    StaticArray<int, 5> myIntArray;
    cout << "数组大小为: " << myIntArray.getSize() << endl;

    myIntArray.set(0, 100);
    myIntArray.set(4, 500);
    myIntArray.set(5, 999); // 故意越界测试

    cout << "arr[0] = " << myIntArray.get(0) << endl;
    cout << "arr[4] = " << myIntArray.get(4) << endl;

    return 0;
}

```

注意事项:

1. 传入的必须是**编译期常量**
2. 非类型参数在模板内部是**只读的**（常量）
3. 不同的非类型参数值，会生成**完全不同的**类，不能互相赋值

类模板与派生

类模板从类模板派生

模板——>模板

```

#include <iostream>
using namespace std;

// 基类模板
template <typename T>
class Base {
protected:
    T value;

public:
    Base(T v) : value(v) {}

    void showBase() {
        cout << "Base value = " << value << endl;
    }
};

// 派生类模板
template <typename T>
class Derived : public Base<T> { //表示用T初始化Base类
private:
    T extraValue;

public:
    Derived(T v1, T v2) : Base<T>(v1), extraValue(v2) {}

    void showDerived() {
        cout << "Derived extraValue = " << extraValue << endl;
    }
};

int main() {
    Derived<int> d1(10, 20);
    d1.showBase();
    d1.showDerived();

    Derived<double> d2(3.14, 6.28);
    d2.showBase();
    d2.showDerived();

    return 0;
}

```

类模板从模板类派生

类——>模板

```
#include <iostream>
using namespace std;

// 基类模板
template <typename T>
class Base {
protected:
    T value;

public:
    Base(T v) : value(v) {
        cout << "Base<T> constructor, value = " << value << endl;
    }

    void showBase() {
        cout << "Base value = " << value << endl;
    }
};

// 类模板从模板类派生
// 派生类是模板，基类是 Base<int>（模板类）
template <typename U>
class Derived : public Base<int> {
private:
    U data;

public:
    Derived(int baseVal, U d) : Base<int>(baseVal), data(d) {
        cout << "Derived<U> constructor, data = " << data << endl;
    }

    void showDerived() {
        cout << "Derived data = " << data << endl;
    }
};

int main() {
    Derived<double> d1(100, 3.14);
    d1.showBase();
}
```

```

d1.showDerived();

cout << "----" << endl;

Derived<string> d2(200, "hello");
d2.showBase();
d2.showDerived();

return 0;
}

```

类模板从普通类派生

```

#include <iostream>
using namespace std;

// 普通基类
class Base {
protected:
    int id;

public:
    Base(int i) : id(i) {
        cout << "Base constructor, id = " << id << endl;
    }

    void showBase() {
        cout << "Base id = " << id << endl;
    }
};

// 类模板从普通类派生
template <typename T>
class Derived : public Base {
private:
    T data;

public:
    Derived(int i, T d) : Base(i), data(d) {
        cout << "Derived<T> constructor, data = " << data << endl;
    }

    void showDerived() {

```



```

        cout << "Derived data = " << data << endl;
    }
};

int main() {
    Derived<int> d1(100, 999);
    d1.showBase();
    d1.showDerived();

    cout << "---" << endl;

    Derived<string> d2(200, "hello");
    d2.showBase();
    d2.showDerived();

    cout << "---" << endl;

    Derived<double> d3(300, 3.14);
    d3.showBase();
    d3.showDerived();

    return 0;
}

```

普通类从模板类派生

```

#include <iostream>
using namespace std;

// 类模板（基类）
template <typename T>
class Container {
protected:
    T data;

public:
    Container(T d) : data(d) {
        cout << "Container<T> constructor, data = " << data << endl;
    }

    void showData() {
        cout << "Data = " << data << endl;
    }
}

```

```
};

// 普通类从模板类派生
class IntContainer : public Container<int> {
private:
    string name;

public:
    IntContainer(int d, string n) : Container<int>(d), name(n) {
        cout << "IntContainer constructor, name = " << name << endl;
    }

    void showInfo() {
        cout << "Name: " << name << endl;
        showData();
    }
};

int main() {
    IntContainer obj(100, "MyContainer");
    obj.showInfo();

    return 0;
}
```

类模板与友元

函数、类、类的成员函数作为类模板的友元

```
#include <iostream>
#include <string>
using namespace std;

// ===== 前置声明 =====
template <typename T>
class Box;

// 普通函数（将作为友元）
template <typename T>
void printBox(const Box<T>& b);

// 普通类（将作为友元类）
class BoxInspector {
```

```

public:
    template <typename T>
    void inspect(const Box<T>& b);
};

// 另一个类（它的成员函数将作为友元）
class BoxManager {
public:
    template <typename T>
    void manage(const Box<T>& b);
};

// ===== 类模板 Box =====
template <typename T>
class Box {
private:
    T data;
    int id;

public:
    Box(T d, int i) : data(d), id(i) {}

    // 1. 函数作为友元
    friend void printBox<T>(const Box<T>& b);

    // 2. 类作为友元
    friend class BoxInspector;

    // 3. 类的成员函数作为友元
    friend void BoxManager::manage<T>(const Box<T>& b);
};

// ===== 友元函数的实现 =====

// 1. 普通函数的实现
template <typename T>
void printBox(const Box<T>& b) {
    cout << "[Function Friend] Box ID: " << b.id
         << ", Data: " << b.data << endl;
}

// 2. 友元类的成员函数实现
template <typename T>

```

```

void BoxInspector::inspect(const Box<T>& b) {
    cout << "[Class Friend] Inspecting Box ID: " << b.id
        << ", Data: " << b.data << endl;
}

// 3. 类的成员函数作为友元的实现
template <typename T>
void BoxManager::manage(const Box<T>& b) {
    cout << "[Member Function Friend] Managing Box ID: " << b.id
        << ", Data: " << b.data << endl;
}

// ===== 主函数 =====
int main() {
    Box<int> b1(100, 1);
    Box<string> b2("hello", 2);
    Box<double> b3(3.14, 3);

    cout << "=== 测试函数友元 ===" << endl;
    printBox(b1);
    printBox(b2);
    printBox(b3);

    cout << "\n=== 测试类友元 ===" << endl;
    BoxInspector inspector;
    inspector.inspect(b1);
    inspector.inspect(b2);
    inspector.inspect(b3);

    cout << "\n=== 测试成员函数友元 ===" << endl;
    BoxManager manager;
    manager.manage(b1);
    manager.manage(b2);
    manager.manage(b3);

    return 0;
}

```

类模板与静态成员变量

- 每个不同的模板实例都有自己独立的静态成员

```

#include <iostream>
using namespace std;

template <typename T>
class Box {
private:
    T data;
    static int count; // 静态成员变量声明

public:
    Box(T d) : data(d) {
        count++;
        cout << "Box<T> created, count = " << count << endl;
    }

    ~Box() {
        count--;
    }

    static int getCount() {
        return count;
    }

    void show() {
        cout << "Data = " << data << ", Count = " << count << endl;
    }
};

// 静态成员变量定义（必须在类外定义）
template <typename T>
int Box<T>::count = 0;

int main() {
    cout << "=== 创建 Box<int> 对象 ===" << endl;
    Box<int> b1(10);
    Box<int> b2(20);
    Box<int> b3(30);
    cout << "Box<int> count = " << Box<int>::getCount() << endl;

    cout << "\n=== 创建 Box<double> 对象 ===" << endl;
    Box<double> d1(3.14);
    Box<double> d2(6.28);
    cout << "Box<double> count = " << Box<double>::getCount() << endl;
}

```

```
cout << "\n=== 再次查看计数 ===" << endl;  
cout << "Box<int> count = " << Box<int>::getCount() << endl;  
cout << "Box<double> count = " << Box<double>::getCount() << endl;  
  
return 0;  
}
```